

TRAE IDE 10 大热门 MCP Server 推荐

本文作者：Jiaqi，TRAE 技术文档工程师

摘要：通过引入合适的 MCP Server，智能体可以深度参与到日常开发流程中，协助完成项目文件读取、官方文档获取、浏览器自动化、代码仓库管理，以及跨会话的上下文维护等任务。

本文将基于真实开发场景，介绍 TRAE IDE 中常用的 10 个 MCP Server，并梳理了它们的核心能力、使用场景与可用工具，帮助你在不同开发阶段选择合适的 MCP Server，从而提升你的日常开发效率。

一、MCP 介绍

Model Context Protocol (MCP) 是一种协议，它允许大型语言模型（LLMs）访问自定义的工具和服务。TRAE 中的智能体作为 MCP 客户端可以选择向 MCP Server 发起请求，以使用它们提供的工具。你可以自行添加 MCP Server，并添加到**自定义的智能体**中来使用。

在 TRAE IDE 中，MCP Server 支持三种传输类型：stdio 传输、SSE 传输、Streamable HTTP 传输。

二、概览

以下是在 TRAE IDE 中的 10 款热门 MCP Server。

MCP Server	简介
Context7	Context7 MCP Server 提供面向 AI 模型的文档检索与上下文注入能力，可实时获取官方文档的最新内容和指定版本的代码示例。
Puppeteer	Puppeteer MCP Server 提供浏览器自动化能力，使 LLM 能够在真实的浏览器环境中与网页进行交互、截取屏幕截图，并执行 JavaScript。
Sequential Thinking	Sequential Thinking MCP Server 通过结构化的思维流程，为动态且具反思性的问题求解提供工具。
GitHub	GitHub MCP Server 基于 GitHub API，允许 LLM 直接访问并管理 GitHub 上的仓库、代码、用户、Issue 与 Pull Request。
Figma AI Bridge	Figma AI Bridge MCP Server 针对设计到实现阶段，提供查看、分析和提取 Figma 设计数据的能力，帮助 LLM 理解设计的结构并辅助精确还原设计稿中内容。
Playwright	

	Playwright MCP Server 基于 Playwright 提供浏览器自动化能力，使 LLM 能够在真实的浏览器环境中与网页交互、截取屏幕截图、生成测试代码、抓取网页内容，并执行 JavaScript。
Memory	Memory MCP Server 通过本地知识图谱（Knowledge Graph）持久化记忆，使 LLM 能够跨会话保留用户相关的上下文信息。
Excel	Excel MCP Server 用于读取 Microsoft Excel 文件中的电子表格数据，或向其中写入数据。
File System	File System MCP Server 提供基于文件系统的文件读取能力。
Chrome DevTools MCP	Chrome DevTools MCP 让 AI 智能体能够直接控制并深入检查 Chrome 浏览器。它向 AI 开放了 Chrome DevTools 的全部能力，使其能更精准、高效地完成网页自动化测试、故障排查及性能分析等任务。

接下来将为你详细介绍每一个 MCP Server 的功能和使用场景，你可以根据自己的使用场景选择合适的 MCP Server。

三、MCP 具体介绍

Context7

Context7 MCP Server 提供面向 AI 模型的文档检索与上下文注入能力，可实时获取产品官方文档的最新内容和指定版本的代码示例，确保模型在回答问题、生成代码或提供方案时，基于最新的官方信息。

- 核心功能
 - **官方文档实时检索：**直接从官方文档源获取内容，而非依赖模型训练阶段的静态知识，支持检索最新或指定版本的 API 参考与示例代码。
 - **文档上下文注入：**将检索到的文档内容直接注入到 LLM 的上下文中，使模型在“已阅读官方文档”的前提下进行回答或生成代码。
 - **库与文档的标准化解析：**将模糊的库名映射为 Context7 可识别的库 ID，确保后续可准确查询到目标文档。
- 使用场景
 - **API 开发：**获取最新框架或 SDK 的 API 定义与示例，避免误用已废弃或不存在的接口。
 - **配置与脚本编写：**编写 Cloudflare Workers、中间件，或构建、部署相关配置时，确保配置项和字段名称与官方文档中的内容一致。
 - **代码生成与重构：**在生成或重构代码时，确保实现方式符合当前的官方推荐做法，减少因文档过时导致的返工。

- **问题排查与最佳实践查询：** 基于官方文档获取错误说明、使用限制和推荐方案，提高问题定位与解决的准确性。
- **工具**

Context7 MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
resolve-library-id	将通用的库名称解析为 Context7 兼容的库 ID。
query-docs	使用 Context7 兼容的库 ID 获取指定库的文档。

Puppeteer

Puppeteer MCP Server 提供浏览器自动化能力，使 LLM 能够在真实的浏览器环境中与网页进行交互、截取屏幕截图，并执行 JavaScript。

- **核心功能**
 - **浏览器控制自动化：** 在真实浏览器中进行页面导航，支持点击、悬停、表单填写等基础网页交互操作，模拟用户行为。
 - **JavaScript 执行：** 直接在浏览器控制台中执行 JavaScript，既能读取当前页面的各类状态信息，也能完成计算任务或触发页面的内置逻辑。
 - **页面捕获：** 对整个页面或指定元素进行截图，为 AI 模型提供直观、可验证的页面渲染结果。
 - **控制台日志监控：** 获取浏览器控制台的输出信息，包括页面脚本产生的所有 console 日志，辅助调试与问题定位。
- **使用场景**
 - **网页功能验证与调试：** 验证网页交互功能是否符合预期效果，同时结合捕获的控制台日志，辅助开发人员快速定位并调试前端问题。
 - **页面状态与渲染检查：** 通过截图功能确认网页的 UI 渲染效果是否达标，同时可对比不同操作前后的页面变化，验证操作对页面的影响。
- **工具**

Puppeteer MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
puppeteer_navigate	在浏览器中导航到任意 URL。
puppeteer_screenshot	对整个页面或指定元素进行截图。

puppeteer_click	点击页面中的元素。
puppeteer_hover	将鼠标悬停在页面元素上。
puppeteer_fill	填写输入框。
puppeteer_select	选择带有 SELECT 标签的元素。
puppeteer_evaluate	在浏览器控制台中执行 JavaScript。

Sequential Thinking

Sequential Thinking MCP Server 通过结构化的思维流程，为动态且具反思性的问题求解提供工具。

- **核心功能**
 - 将复杂问题拆解为可管理的步骤。
 - 随着理解加深，对思路进行修订和完善。
 - 在不同的推理路径之间进行分支探索。
 - 动态调整整体思考步骤的数量。
 - 生成并验证解决方案假设。
- **使用场景**
 - 拆解复杂问题，并按步骤逐步解决。
 - 需要预留修改空间的规划与设计过程。
 - 可能需要中途调整方向的分析任务。
 - 一开始难以完全明确问题范围的场景。
 - 需要在多个步骤中持续保持上下文的任务。
 - 需要过滤无关信息、聚焦关键信息的情况。
- **工具**

Sequential Thinking MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
sequential_thinking	为问题求解和分析提供细致的、逐步推进的思考过程。

GitHub

GitHub MCP Server 基于 GitHub API，允许 LLM 直接访问并管理 GitHub 上的仓库、代码、用户、Issue 与 Pull Request。

该 MCP Server 支持的所有操作**仅作用于** GitHub 上的远程资源：所有文件操作均发生在 GitHub 仓库中，通过 Commit 与 Pull Request 提交，**不会读取或修改用户本地的文件系统。**

核心功能

- 仓库与文件管理：**支持全面管理代码库，包括仓库的创建、Fork、搜索及分支管理；同时具备精细的文件操作功能，允许直接读取内容、创建或更新文件以及批量推送代码。
- Issue 追踪：**聚焦于项目进度的管理与协同，支持对 Issue 进行创建、筛选、状态更新及评论。
- Pull Request 协作：**涵盖代码合并的全生命周期，从发起 Pull Request、查看变更详情、同步分支最新改动，到最终合并代码。
- 代码评审与信息检索：**支持发起和管理代码评审，获取评审意见；同时提供强大的搜索能力，可快速定位代码片段、用户或相关评论。

使用场景

- AI 辅助代码开发：**自动执行从代码修改、分支创建到变更提交的全流程。在遵循版本控制规范的前提下，实现开发需求的快速落地与代码历史的清晰记录。
- 自动化协作工作流：**全面接管 Issue 追踪与 Pull Request 管理。自动完成任务创建、上下文补充、评审及分支合并，减少人工操作成本。
- 项目调研与分析：**深度解析仓库的架构、代码逻辑及变更历史。结合全局检索能力，快速定位核心信息并梳理项目脉络，然后生成调研报告。
- 智能的仓库管理：**将 AI 作为团队协作的一部分，执行重复性的 GitHub 操作，提升个人或团队的开发效率。

工具

GitHub MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
create_or_update_file	在仓库中创建或更新单个文件。
push_files	在一次提交中推送多个文件。
search_repositories	搜索 GitHub 仓库。
create_repository	创建新的 GitHub 仓库。
get_file_contents	获取文件或目录内容。

create_issue	创建新的 Issue。
create_pull_request	创建新的 Pull Request。
fork_repository	Fork 一个仓库。
create_branch	创建新分支。
list_issues	列出并筛选仓库 Issue。
update_issue	更新已有 Issue。
add_issue_comment	为 Issue 添加评论。
search_code	在 GitHub 上搜索代码。
search_issues	搜索 Issue 和 Pull Request。
search_users	搜索 GitHub 用户。
list_commits	获取仓库某个分支的提交记录。
get_issue	获取仓库中指定 Issue 的内容。
get_pull_request	获取指定 Pull Request 的详情。
list_pull_requests	列出并筛选仓库的 Pull Request。
create_pull_request_review	为某个 Pull Request 创建评审。
merge_pull_request	合并 Pull Request。
get_pull_request_files	获取某个 Pull Request 中变更的文件列表。
get_pull_request_status	获取某个 Pull Request 的所有状态检查的汇总状态。
update_pull_request_branch	使用 base 分支的最新更改更新某个 Pull Request 分支（等同于 GitHub 中的“Update branch”按钮）。
get_pull_request_comments	获取某个 Pull Request 的评审评论。
get_pull_request_reviews	获取某个 Pull Request 的评审记录。

Figma AI Bridge

Figma AI Bridge MCP Server 针对设计到实现阶段，提供查看、分析和提取 Figma 设计数据的能力，帮助 LLM 理解你的设计思路并辅助你精确还原设计稿中的内容。

● 核心功能

- **Figma 设计解析：**获取 Figma 文件或指定节点的布局与结构信息。在无法直接获得节点 ID 的情况下，仍可分析整个设计文件。
- **设计资源下载：**根据图像或图标节点 ID，自动下载设计中使用的 SVG / PNG 图片，便于在实现阶段直接复用设计资产。
- **为 AI 提供可理解的设计上下文：**将 Figma 中的设计信息转换为 AI 可消费的数据，为后续的代码生成、样式还原或布局分析提供依据。

● 使用场景

- **设计还原与前端实现：**辅助 AI 理解设计的结构，提高实际实现与设计稿的一致性。
- **设计资产自动提取：**自动下载图标、图片等资源，减少手动操作。
- **设计到代码的自动转换：**为代码生成提供设计思路相关的准确上下文，作为从设计到实现链路中的关键一环。
- **Agent 驱动的 UI 分析与实现：**让 AI 在理解设计思路的基础上分析 UI 布局并思考实现方案。

● 工具

Figma AI Bridge MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
download_figma_images	根据图像或图标节点的 ID，下载 Figma 文件中使用的 SVG 和 PNG 图片。
get_figma_data	当无法获取节点 ID 时，用于获取整个 Figma 文件的布局信息。同时支持获取某个 Figma 文件或文件中指定节点的相关信息。

Playwright

Playwright MCP Server 基于 Playwright 提供浏览器自动化能力，使 LLM 能够在真实的浏览器环境中与网页交互、截取屏幕截图、生成测试代码、抓取网页内容，并执行 JavaScript。

相较于基础浏览器自动化，它进一步扩展了测试代码生成、网络请求控制与多设备模拟等能力，适合更复杂、结构化的网页测试。

● 核心功能

- **自动化浏览器控制**
 - 支持页面导航、点击、悬停、表单填写、拖拽、键盘操作等交互。
 - 覆盖普通 DOM 以及 iframe 场景。

- **代码生成与测试录制 (Codegen)**
 - 支持开启代码生成会话，记录浏览器操作。
 - 自动生成可复用的 Playwright 测试代码文件。
- **页面内容捕获**
 - 对页面或指定元素进行截图。
 - 提取页面可见文本或 HTML 内容。
 - 支持将页面保存为 PDF。
- **JavaScript 执行与控制台调试**
 - 在浏览器上下文中执行 JavaScript。
 - 获取并过滤浏览器控制台日志，用于调试与分析。
- **网络请求与响应控制**
 - 主动发起 HTTP 请求。
 - 支持等待并断言特定网络响应，便于接口级验证。
- **多设备与浏览器环境模拟**
 - 调整浏览器视口大小或使用设备预设。
 - 内置 143+ 设备模型，提供正确的 User-Agent 与触控模拟，支持自定义 User-Agent。
- **使用场景**
 - **自动化测试与测试代码生成**
 - 执行由 AI 驱动的网页测试并录制真实操作，生成 Playwright 测试脚本。
 - 回归测试、端到端测试。
 - **复杂网页交互验证**
 - 验证页面在不同设备、分辨率和 User-Agent 下的交互逻辑。
 - 处理 iframe、文件上传、标签页切换等复杂交互。
 - **请求级调试与验证**
 - 同时验证前端交互与后端接口响应。
 - 适合需要精确控制请求与响应的调试场景。
 - **Agent 驱动的网页操作与信息采集**
 - 抓取可见内容或结构化 HTML。
 - 在真实浏览器上下文中完成端到端任务。

- **工具**

Playwright MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
start_codegen_session	开始一个新的代码生成会话，用于记录 Playwright 操作。
end_codegen_session	结束代码生成会话并生成测试文件。
get_codegen_session	获取关于代码生成会话的信息。
clear_codegen_session	清除代码生成会话而不生成测试文件。
playwright_navigate	导航到一个 URL。
playwright_screenshot	对当前页面或特定元素进行截图。
playwright_click	点击页面上的元素。
playwright_iframe_click	点击 iframe 中的元素。
playwright_iframe_fill	在页面中的 iframe 里填充某个元素。
playwright_fill	填写输入字段。
playwright_select	使用 Select 标签选择页面上的元素。
playwright_hover	悬停在页面的元素上。
playwright_upload_file	将文件上传到页面中的 input[type="file"] 元素。
playwright_evaluate	在浏览器控制台执行 JavaScript。
playwright_console_logs	检索浏览器的控制台日志（带过滤选项）。
playwright_resize	使用自定义尺寸或设备预设来调整浏览器视口大小。支持 143 种以上的设备预设，包括 iPhone、iPad、各类 Android 设备以及桌面浏览器，并提供正确的 User-Agent 和触控（Touch）模拟。
playwright_close	关闭浏览器并释放所有资源。
playwright_get	执行 HTTP GET 请求。
playwright_post	执行 HTTP POST 请求。
playwright_put	执行 HTTP PUT 请求。
playwright_patch	执行 HTTP PATCH 请求。
playwright_delete	执行 HTTP DELETE 请求。
playwright_expect_response	请求 Playwright 开始等待某个 HTTP 响应。该工具只会启动等待操作，但不会阻塞或等待该操作完成。

playwright_assert_response	等待并校验之前已发起的 HTTP 响应等待操作。
playwright_custom_user_agent	为浏览器设置自定义 User-Agent。
playwright_get_visible_text	获取当前页面的可见文本内容。
playwright_get_visible_html	获取当前页面的 HTML 内容。默认情况下，输出结果会移除所有 <script> 标签，除非显式将 removeScripts 设置为 false。
playwright_go_back	在浏览器历史中后退。
playwright_go_forward	在浏览器历史中前进。
playwright_drag	将元素拖动到目标位置。
playwright_press_key	按下键盘键。
playwright_save_as_pdf	将当前页面保存为 PDF 文件。
playwright_click_and_switch_tab	点击一个链接并切换到新打开的标签页。

Memory

Memory MCP Server 通过本地知识图谱（Knowledge Graph）持久化记忆，使 LLM 能够跨会话保留用户相关的上下文信息。

它的**核心目标是将零散、非结构化的用户信息转化为可结构化、可查询、可演化的长期记忆，并在后续对话中被持续维护和利用。**

Memory MCP Server 采用知识图谱作为记忆模型，由以下三类核心概念共同构成一个可扩展的记忆网络：

概念	描述
Entity	知识图谱中的主要节点，表示一个具体、可识别的对象。
Relation	用于定义 Entity 之间的有向连接。它们始终以主动语态存储，用于描述 Entity 之间的交互方式或相互关系。
Observation	关于某个 Entity 的离散信息片段，可不断新增、修改或删除。

- 核心功能

- **结构化、持久化记忆：**将用户相关信息以 Entity-Relation-Observation 的形式存储。存储的信息会跨会话持续存在，不随单次对话结束而丢失。
- **记忆的增量演化：**支持向已有 Entity 不断添加新的 Observation，以及创建和调整 Entity 之间的 Relation。
- **查询与维护记忆：**支持按条件搜索节点。可读取整个知识图谱，便于调试和理解记忆。
- **清理记忆：**支持删除指定 Entity 及其关联的 Relation；支持删除指定 Observation 或 Relation，避免记忆污染。
- **使用场景**
 - **跨会话用户记忆：**记住你的偏好、背景、项目状态等信息，从而在后续对话中提供更连续、一致的体验。
 - **Agent 的长期状态管理：**让 AI Agent 拥有可演化的“内部状态”，支撑长期任务与多轮协作。
 - **复杂上下文的结构化管理：**避免将大量历史信息堆叠在 Prompt 中，提高上下文可控性与可维护性。
 - **可解释的 AI 记忆系统：**通过知识图谱，清晰展示模型所记住的信息，便于调试和人工干预。

- **工具**

Memory MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
create_entities	在知识图谱中创建多个新 Entity。
create_relations	在 Entity 之间创建多条新关系。
add_observations	向已有 Entity 添加新 Observation。
delete_entities	删除 Entity 及其所有 Relation。
delete_observations	从 Entity 中删除指定的 Observation。
delete_relations	从知识图谱中删除指定的 Relation。
read_graph	读取整个知识图谱。
search_nodes	根据查询条件搜索节点。
open_nodes	按名称获取指定节点。

Excel MCP Server 用于读取 Microsoft Excel 文件中的电子表格数据，或向其中写入数据。

● 核心功能

- 读取 / 写入文本值
- 读取 / 写入公式
- 创建新的工作表
- (仅 Windows) 实时编辑
- (仅 Windows) 为工作表截图

● 使用场景

- **自动化数据处理：** 批量读取、整理和写入 Excel 数据，自动生成结构化表格与计算公式。
- **报表生成：** 自动生成分析报表或业务数据表。
- **办公自动化与 Agent 流程：** 作为 AI Agent 操作办公文件的重要接口，可以与其他 MCP Server 协同使用。
- **可视化结果校验（Windows）：** 通过截图方式确认表格的布局与内容是否符合预期。

● 工具

Excel MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
excel_describe_sheets	列出指定 Excel 文件中的所有工作表信息。
excel_read_sheet	以分页方式读取 Excel 工作表中的数据。
excel_screen_capture	(仅 Windows) 以分页方式为 Excel 工作表截图。
excel_write_to_sheet	向 Excel 工作表写入数据。
excel_create_table	在 Excel 工作表中创建表格。
excel_copy_sheet	将现有工作表复制为一个新的工作表。
excel_format_range	为 Excel 工作表中的单元格设置样式格式。

File System

File System MCP Server 提供基于文件系统的文件读取能力。

● 主要功能

- 通过 MCP 实现无缝的文件读取。
- 命令行式的 API Key 配置。
- 使用场景
 - 文档与配置读取：**在对话或自动化流程中直接读取项目文档、配置文件或说明文件，为模型提供准确的上下文。
 - 代码与资源分析：**读取源代码、脚本或资源文件，辅助进行代码理解、审查或问题定位。
 - 工作流集成：**将文件读取能力集成到现有流程中，减少手动复制粘贴文件内容，提高自动化水平。
- 工具

File System MCP Server 为 LLM 提供以下可调用的工具：

工具	描述
read-file	从文件系统读取文件。

Chrome DevTools MCP

Chrome DevTools MCP 让 AI 智能体能够直接控制并深入检查 Chrome 浏览器。它向 AI 开放了 Chrome DevTools 的全部能力，使其能更精准、高效地完成网页自动化测试、故障排查及性能分析等任务。

- 主要功能
 - 浏览器自动化：**通过 Chrome DevTools Protocol 在 Chrome 中自动化各类操作，并自动等待结果。
 - 性能洞察：**使用 Chrome DevTools 记录性能追踪（traces）数据，并提取可执行的性能优化策略。
 - 浏览器调试：**分析网络请求、截取屏幕截图，并检查浏览器控制台信息。
- 使用场景
 - 浏览器相关操作自动化：**由 AI 直接控制 Chrome 来执行页面交互（导航、点击、输入、等待等），实现稳定、可复现的自动化流程。
 - 前端调试与问题排查：**访问控制台日志、网络请求和页面状态，辅助定位前端错误、接口异常等问题。
 - 性能分析与优化：**记录并分析性能追踪数据，获取性能洞察和 Core Web Vitals 指标，用于发现并处理页面的性能瓶颈。

● 工具

Chrome DevTools MCP 为 LLM 提供以下可调用的工具：

工具	描述
click	点击指定的元素。
close_page	根据页面索引关闭页面。最后一个打开的页面不能被关闭。
drag	将一个元素拖拽到另一个元素上。
emulate	在选中的页面上模拟多种特性。
evaluate_script	在当前选中的页面中执行一个 JavaScript 函数，并将响应以 JSON 形式返回，使返回值可被 JSON 序列化。
fill	在输入框或文本区域中输入文本，或从 <select> 元素中选择一个选项。
fill_form	一次性填写多个表单元素。
get_console_message	根据 ID 获取一条控制台消息。你可以通过调用 list_console_messages 获取所有消息。
get_network_request	根据可选的 reqid 获取一条网络请求；如果省略，则返回 DevTools Network 面板中当前选中的请求。
handle_dialog	如果打开了浏览器对话框，使用该命令进行处理。
hover	将鼠标悬停在提供的元素上。
list_console_messages	列出自上次导航以来，当前选中页面的所有控制台消息。
list_network_requests	列出自上次导航以来，当前选中页面的所有网络请求。
list_pages	获取浏览器中打开的页面列表。
navigate_page	将当前选中的页面导航到一个 URL。
new_page	创建一个新的页面。
performance_analyze_insight	针对追踪记录结果中高亮显示的某个 Performance Insight，提供更详细的信息。
performance_start_trace	在选中的页面上开始性能追踪记录。该记录可用于查找性能问题并获取改进页面性能的洞察，同时还会报告页面的 Core Web Vital（CWV）分数。
performance_stop_trace	在所选中页面上，停止活跃的性能追踪记录。
press_key	

	按下某个按键或组合键。当无法使用其他输入方式（如 fill()）时使用（例如键盘快捷键、导航键或特殊组合键）。
resize_page	调整所选中页面的窗口的大小，使页面具有指定的尺寸。
select_page	选择一个页面，作为后续工具调用的上下文。
take_screenshot	对页面或元素进行截图。
take_snapshot	基于 a11y（Accessibility）树，对当前选中的页面生成文本快照。快照会列出页面元素及其唯一标识（uid）。始终使用最新快照。 相比截图，优先使用快照。快照会指示 DevTools Elements 面板中选中的元素（如果有）。
upload_file	通过提供的元素上传文件。
wait_for	等待指定文本出现在选中的页面上。

如何添加这些 MCP Server?

你可以直接从 TRAE IDE 内置的 MCP 市场添加。

1. 进入 MCP 设置中心：
 - a. **IDE 模式**：在 IDE 模式界面中，点击界面右上角的 **设置** 图标，进入设置中心。
 - b. **SOLO 模式**：在 SOLO 模式界面中，点击对话面板右上角的 **设置** 图标，进入设置中心。
2. 在左侧导航栏中，选择 **MCP**，打开 **MCP** 窗口。
3. 在 **MCP** 窗口的右上角，点击 **添加 > 从市场添加**。若你是首次添加 MCP Server，还可以直接点击窗口中部的 **从市场添加** 按钮。

Q 搜索

**Puppeteer** Local

Enables browser automation and web scraping with Puppeteer, allowing LLMs to interact ...

**PostgreSQL** Local

Provides read-only access to PostgreSQL databases, enabling LLMs to inspect database ...

**GitHub** Local

Integrates with the GitHub API, enabling repository management, file operations, issue tra...

**Figma AI Bridge** Local

Offers tools to view, comment on, and analyze Figma designs, ensuring precise implemen...



4. 在 MCP 市场中找到所需的 MCP Server。

5. 点击右侧的 + 按钮。

6. 在弹窗中填入 MCP Server 的配置信息。

- 对于标记为 “Local” 的 MCP Server，需要在本地安装 NPX 或 UVX 后才能使用。
- 配置内容中的 env 信息（例如 API Key、Token、Access Key 等字段）需替换为真实信息。

7. 点击 **确认** 按钮，即代表配置成功。

结语

以上是在 TRAE IDE 中的 10 个 热门 MCP Server 的详细介绍，快来立即体验使用吧！

更多信息请关注TRAE企业版服务号👉



微信搜一搜



TRAE企业版